

IEOR 6616: Convex Optimization

Programming Project: Computational Optimal Transport

Instructor: Bento Natura

Format: Teams of 2 (solo is permitted); both names and UNIs on report (and/or README)
Released: April 8, 2026
Due: May 15, 2026 (11:59 PM ET)
Submission: GitHub repository + PDF report (3 pages max, excl. figures)
Language: Python (NumPy/SciPy required; CVXPY, OR-Tools, PyTorch, or JAX permitted)

1 Overview

Optimal transport (OT) asks: given two probability distributions, what is the cheapest way to move mass from one to the other? Monge posed this in 1781; Kantorovich reformulated it as a linear program in 1942, as part of his broader work on linear programming for resource allocation (1975 Nobel Prize in Economics, shared with Koopmans).

The optimal transport cost defines a metric on distributions, the *Wasserstein distance*. Unlike KL divergence, which requires the distributions to share support, the Wasserstein distance is finite for any two distributions and reflects the geometry of the underlying space. These properties have been known for a long time, but the computational cost of solving the LP limited practical use. Cuturi [2] observed that entropic regularization makes the problem solvable by the Sinkhorn algorithm (a matrix scaling method from 1964), reducing the cost to matrix-vector multiplies. This made OT practical as a subroutine in machine learning. It now appears in generative modeling (flow matching [5]), computational biology (tracking cell populations over time), fairness (measuring distributional inequity), and logistics (the original Monge problem).

In this project, you will solve OT using algorithms from this course: LP solvers, ADMM, and first-order primal-dual methods. You will see how changing the regularizer (from quadratic to entropic) changes the geometry of the algorithm, turning Euclidean projections (ADMM) into KL projections (the Sinkhorn algorithm).

The relevant formulations and algorithms are described below. **Your job is to find the right solvers, run the experiments, and discuss the results.** Certain implementations are marked as *optional* (e.g., writing your own PDHG loop or Sinkhorn); these are for **extra credit**.

2 Policy on LLM Usage

You may use LLMs freely for coding. Your report must contain a brief section describing how you used them.

3 Setup

Test instances. Throughout, use **random point clouds** in $\mathbb{R}^2$: sample $m = n$ source points from $\mathcal{N}(\mu_1, \Sigma_1)$ and n target points from $\mathcal{N}(\mu_2, \Sigma_2)$, with $\mu_1 \neq \mu_2$ and non-diagonal covariances Σ_1, Σ_2 (i.e., not axis-aligned). Use uniform weights $\mathbf{a} = \mathbf{b} = \frac{1}{n}\mathbf{1}$ and squared Euclidean cost $C_{ij} = \|x_i - y_j\|_2^2$. Set a random seed for reproducibility.

Notation. A matrix $P \in \mathbb{R}_{\geq 0}^{m \times n}$ with row sums $P\mathbf{1} = \mathbf{a}$ and column sums $P^\top \mathbf{1} = \mathbf{b}$ is called a **coupling** (or transport plan): P_{ij} is the amount of mass moved from source i to target j . The set of all couplings is the **transportation polytope** $U(\mathbf{a}, \mathbf{b}) = \{P \in \mathbb{R}_{\geq 0}^{m \times n} : P\mathbf{1} = \mathbf{a}, P^\top \mathbf{1} = \mathbf{b}\}$. The **Frobenius norm** is $\|P\|_F = (\sum_{ij} P_{ij}^2)^{1/2}$, and $\langle C, P \rangle = \sum_{ij} C_{ij} P_{ij}$.

Two regularizations. Besides the LP, we study two regularized variants of OT. Both make the problem strictly convex (guaranteeing a unique solution) and recover the LP optimum as the regularization vanishes:

- **Quadratic** ($\frac{\lambda}{2}\|P\|_F^2$): the optimal solution is soft-thresholded, $P_{ij}^* = \max(0, (f_i + g_j - C_{ij})/\lambda)$ for dual variables f, g , so entries can be exactly zero. The problem is a QP, and the natural algorithm (ADMM) uses Euclidean projections: additive updates, clipping to zero.
- **Entropic** ($\lambda \sum_{ij} P_{ij} \log P_{ij}$): the optimal solution has the exponential form $P_{ij}^* = \exp((f_i + g_j - C_{ij})/\lambda)$, which is strictly positive everywhere. The natural algorithm (Sinkhorn) uses KL projections: multiplicative rescaling of rows and columns.

Notice how the regularizer determines the algorithm: quadratic $\rightarrow$ Euclidean projections (ADMM); entropic $\rightarrow$ KL projections (Sinkhorn).

4 Part 1: The LP: Three Solver Paradigms

The **Kantorovich problem** is the LP:

$$W_{\text{LP}} = \min_{P \in U(\mathbf{a}, \mathbf{b})} \langle C, P \rangle. \quad (1)$$

Its **Lagrangian dual** (introduce multipliers f_i for $P\mathbf{1} = \mathbf{a}$ and g_j for $P^\top \mathbf{1} = \mathbf{b}$, minimize over $P \geq 0$) is:

$$\max_{f \in \mathbb{R}^m, g \in \mathbb{R}^n} \mathbf{a}^\top f + \mathbf{b}^\top g \quad \text{s.t.} \quad f_i + g_j \leq C_{ij} \quad \forall i, j. \quad (2)$$

For decades, two paradigms have dominated LP solving: the simplex method (Dantzig, 1947), which walks along vertices of the feasible polytope, and interior point methods (Karmarkar, 1984), which cut through the interior via Newton steps on a barrier-penalized objective. Both produce high-accuracy solutions but require expensive per-iteration work: pivot operations for simplex, linear system solves for IPM.

More recently, first-order primal-dual methods have emerged as a serious alternative. The starting point is *dual ascent*, which alternates between minimizing the Lagrangian over the primal variable and taking a gradient step on the dual. PDHG (Chambolle–Pock [4]) modifies this in two ways: (i) the exact primal minimization is replaced by a single proximal step, and (ii) an extrapolation step on the primal variable is added to ensure convergence. Neither modification is something

we covered in class, but both build directly on ideas we did cover. The result: each iteration is $O(mn)$ element-wise arithmetic, with no linear algebra. The trade-off: iterations are cheap and parallelizable, but many are needed, and the solution is only approximate.

Google's PDLP solver [3] showed in 2021 that with careful engineering (adaptive step sizes, preconditioning, restarts), PDHG can be competitive with simplex and IPM on large-scale LPs. PDLP is now a production solver at Google.

Deliverables

- (a) **Solve with three paradigms.** Using the random point clouds described in the Setup, solve (1) and report solve times for $n \in \{50, 100, 200, 500\}$. Verify correctness on small examples ($n \in \{3, 4\}$). It may be useful to write (1) in standard form ($\min c^\top x$ s.t. $A_{\text{eq}} x = b_{\text{eq}}, x \geq 0$) by vectorizing P and assembling the marginal constraints into a sparse matrix.

- **Simplex:** any LP solver that uses the simplex method (e.g., `scipy.optimize.linprog` with `method='highs-ds'`).
- **Interior point method:** any LP solver that uses an IPM (e.g., `scipy.optimize.linprog` with `method='highs-ipm'`).
- **PDHG / PDLP:** The LP can be written as a saddle point problem

$$\min_{P \geq 0} \max_{f, g} \langle C, P \rangle + f^\top (\mathbf{a} - P\mathbf{1}) + g^\top (\mathbf{b} - P^\top \mathbf{1}),$$

which PDHG solves by alternating a primal proximal step with a dual gradient ascent step:

$$\begin{aligned} P^{k+1} &= \max \left(0, P^k - \tau (C - f^k \mathbf{1}^\top - \mathbf{1} (g^k)^\top) \right), \\ \bar{P}^{k+1} &= 2P^{k+1} - P^k, \\ \begin{pmatrix} f^{k+1} \\ g^{k+1} \end{pmatrix} &= \begin{pmatrix} f^k \\ g^k \end{pmatrix} + \sigma \left(\begin{pmatrix} \bar{P}^{k+1} \mathbf{1} \\ (\bar{P}^{k+1})^\top \mathbf{1} \end{pmatrix} - \begin{pmatrix} \mathbf{a} \\ \mathbf{b} \end{pmatrix} \right). \end{aligned}$$

Each iteration is just element-wise arithmetic and row/column sums. There is no hidden complexity (contrast with simplex pivoting or Newton system solves in IPM).

Use any solver that implements PDHG (e.g., Google's **PDLP** via `pip install ortools` [3]).

Optional (extra credit): implement the PDHG loop yourself in NumPy. Experiment with different step sizes τ, σ and discuss what works and what doesn't. Can you make your implementation competitive with PDLP? What happens if you add restarts or adaptive step sizes? See [6] for an overview of these techniques.

A note on hardware. In this project you will run everything on CPU, where simplex and IPM will likely dominate. But PDHG's real advantage is on GPUs: each iteration is just element-wise arithmetic and row/column sums, operations that map perfectly onto massively parallel hardware. A GPU implementation of the same loop (e.g., replacing `numpy` with `jax.numpy`) can be orders of magnitude faster for large n , which is why Google's PDLP is a production solver for LPs with millions of variables.

Produce a **log-log plot** of time vs. n for all methods (simplex, IPM, PDLP, and optionally your own PDHG).

- (b) **Accuracy.** Simplex and IPM return exact solutions (up to floating-point precision), but PDHG returns an approximate one. For each method, report the **transport cost** $\langle C, P \rangle$ and the **marginal violation** $\|P\mathbf{1} - \mathbf{a}\|_1 + \|P^\top \mathbf{1} - \mathbf{b}\|_1$. Present the timing and accuracy results in a table. How close does PDHG get to the LP optimum?

The LP solution serves as ground truth for the rest of the project: use it to check your results in Sections 5–8.

5 Part 2: Quadratic Regularization

Adding a quadratic penalty gives a strictly convex QP (since $\|P\|_F^2$ is strongly convex):

$$\min_{P \in U(\mathbf{a}, \mathbf{b})} \langle C, P \rangle + \frac{\lambda}{2} \|P\|_F^2. \quad (3)$$

Solve (3) using two algorithms from the course: **ADMM** and an **interior point method**. For example, in CVXPY the solver **OSQP** uses ADMM and the solver **ECOS** uses an IPM; you are free to use other solvers as long as you know which algorithm they run.

ADMM formulation. For reference, the ADMM splitting for (3) introduces an auxiliary $Q \in U(\mathbf{a}, \mathbf{b})$ and the updates with penalty $\rho > 0$ and scaled dual variable U (initialized to zero) are:

$$P_{ij}^{k+1} = \max\left(0, \frac{\rho(Q_{ij}^k - U_{ij}^k) - C_{ij}}{\lambda + \rho}\right), \quad (4)$$

$$Q^{k+1} = \Pi_{U(\mathbf{a}, \mathbf{b})}(P^{k+1} + U^k), \quad (5)$$

$$U^{k+1} = U^k + P^{k+1} - Q^{k+1}. \quad (6)$$

Deliverables

- Solve with ADMM and IPM.** Compare solve times for $n = 200$.
- Regularization path.** Solve for several values of λ (e.g., logarithmically spaced from 10 to 10^{-3}). Plot $\langle C, P_\lambda^* \rangle$ vs. λ and verify convergence to W_{LP} as $\lambda \rightarrow 0$.
- Effect of the ADMM penalty ρ .** Solve (3) for a fixed λ but with different values of ρ . Does convergence speed change? (Most ADMM solvers expose this parameter, e.g., **rho** in OSQP.)

6 Part 3: Entropic Regularization (Sinkhorn)

Adding an entropy penalty to the OT objective gives the **entropically regularized** problem:

$$\min_{P \in U(\mathbf{a}, \mathbf{b})} \langle C, P \rangle + \lambda \sum_{ij} P_{ij} \log P_{ij}. \quad (7)$$

Large λ gives a smooth, diffuse coupling; small λ recovers the LP solution.

Define the **Gibbs kernel** $K_{ij} = e^{-C_{ij}/\lambda}$. One can show that the optimal coupling has the form

$$P^* = \text{diag}(u) K \text{diag}(v)$$

for positive vectors $u \in \mathbb{R}_{>0}^m$, $v \in \mathbb{R}_{>0}^n$. The Sinkhorn algorithm maintains this structure throughout: starting from $v = \mathbf{1}$, it alternates

$$u \leftarrow \frac{\mathbf{a}}{Kv}, \quad v \leftarrow \frac{\mathbf{b}}{K^\top u},$$

where the divisions are elementwise and all coordinates are updated simultaneously. The u -update enforces the row marginal ($P\mathbf{1} = \mathbf{a}$); the v -update enforces the column marginal ($P^\top \mathbf{1} = \mathbf{b}$). At convergence, both hold simultaneously and $P = \text{diag}(u) K \text{diag}(v)$ is the optimal coupling.

This is equivalent to alternating projections of K onto the row-marginal set $\mathcal{C}_1 = \{P \geq 0 : P\mathbf{1} = \mathbf{a}\}$ and the column-marginal set $\mathcal{C}_2 = \{P \geq 0 : P^\top \mathbf{1} = \mathbf{b}\}$, where the projections use the KL divergence instead of the Euclidean distance. Compare with Section 5: ADMM alternates *Euclidean* projections (additive updates, clipping to zero); Sinkhorn alternates *KL* projections (multiplicative updates, row/column rescaling). Same algorithmic template, different geometry, dictated by the regularizer.

Deliverables

- (a) **Run Sinkhorn.** Use any Sinkhorn implementation (e.g., POT: `pip install pot`, function `ot.sinkhorn`).
Optional (extra credit): implement the alternating row/column rescaling yourself. It is a few lines of NumPy. What happens for small λ ? Can you handle it?
 - (b) **Compare with an IPM.** Solve (7) with any interior point or conic solver. In CVXPY, note that `cp.entr(P) = -P log P` is concave, so the objective is `cp.sum(cp.multiply(C, P)) - lambda * cp.sum(cp.entr(P))`. Compare the solve time and solution to Sinkhorn.
 - (c) **Convergence.** For $n = 200$ and $\lambda \in \{1, 0.1, 0.01\}$, plot marginal violation vs. iteration.
 - (d) **Regularization path.** For 15 values of λ from 10 to 0.01 (log-spaced), plot $\langle C, P_\lambda^* \rangle$ vs. λ and verify convergence to W_{LP} .
-

7 Visualization

Think about how to visualize the couplings you computed in Sections 4–6. The coupling P^* encodes how mass moves from source to target, but a raw $n \times n$ matrix is hard to interpret. Come up with a visualization that makes the structure visible.

Use the *same point cloud* and show side by side: the LP coupling, the quadratic coupling at $\lambda = 0.1$, and the Sinkhorn coupling at $\lambda = 1$ and $\lambda = 0.01$. The differences should be obvious. Submit either static figures in the report or an interactive notebook (Jupyter/Colab).

8 Part 4: Gaussian Optimal Transport

When both distributions are Gaussian $\mu_0 = \mathcal{N}(\mu_1, \Sigma_1)$, $\nu_0 = \mathcal{N}(\mu_2, \Sigma_2)$ on $\mathbb{R}^d$, the squared Wasserstein-2 distance has the **Bures–Wasserstein** closed form [5]:

$$W_2^2(\mu_0, \nu_0) = \|\mu_1 - \mu_2\|^2 + \text{tr}\left(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{1/2} \Sigma_2 \Sigma_1^{1/2})^{1/2}\right). \quad (8)$$

Deliverables

- (a) Choose two 2D Gaussians with distinct means and covariances. Compute W_2^2 via (8). (The matrix square root $\Sigma^{1/2}$ can be computed with `scipy.linalg.sqrtm`.)
 - (b) For $n \in \{50, 100, 200, 500\}$: sample n points from each Gaussian, solve the discrete OT problem (using any method from Sections 4–6), and record the transport cost. To reduce variance, repeat with 5–10 different random seeds and plot the **mean** $\pm$ one standard deviation.
 - (c) **Plot** discrete cost vs. n and show convergence to W_2^2 .
 - (d) **Visualization**. For $n = 200$: arrows from source to matched target, overlaid on Gaussian contour ellipses.
-

9 Report (3 pages max, excluding figures)

State which solvers you used and which algorithms they implement. Summarize your results: timing, accuracy, regularization paths. Discuss what you observed. Include all figures with captions (figures do not count toward the page limit). Briefly describe how you used LLMs.

10 Submission

Submit via CourseWorks. Both team members must be listed.

1. **Code**: either a GitHub repository (or .zip) with source code and a README.md, or a Jupyter/Colab notebook. Include names, UNIs, and instructions to reproduce all results.
2. **PDF report** (3 pages max, excl. figures). Both names and UNIs on page 1.

References

- [1] G. Peyré and M. Cuturi. *Computational Optimal Transport*. Foundations and Trends in Machine Learning, 2019. <https://arxiv.org/abs/1803.00567>.
- [2] M. Cuturi. *Sinkhorn Distances: Lightspeed Computation of Optimal Transport*. NeurIPS, 2013.
- [3] D. Applegate et al. *Practical Large-Scale Linear Programming using Primal-Dual Hybrid Gradient*. NeurIPS, 2021.
- [4] A. Chambolle and T. Pock. *A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging*. JMIV, 2011.
- [5] Y. Lipman et al. *Flow Matching for Generative Modeling*. ICLR, 2023.
- [6] H. Lu and J. Yang. *An Overview of GPU-based First-Order Methods for Linear Programming and Extensions*. <https://arxiv.org/abs/2503.09566>, 2025.

You may use any optimization library (CVXPY, SciPy, OR-Tools, POT, etc.). The emphasis is on correct formulation, choosing the right solver, and understanding the results, not on implementing algorithms from scratch.